Programming Techniques 2026–2027

Course exercise 2: distributed Barnes–Hut simulation with MPI

Hannu Parviainen

Universidad de la Laguna

October 19, 2026

Exercise 2: Barnes–Hut simulation with MPI

- The direct sum of Exercise 1 takes a time proportional to N^2.
- The Barnes–Hut algorithm (Lecture 11) treats a distant group of particles as a single point mass at its centre of mass. It takes a time proportional to $N \log N$.
- In this exercise, you turn a serial Barnes–Hut code into a modular program, and distribute the calculation over several processes with MPI.

Starting code:
`lectures/exercise_2/barnes_hut.f90`

- A serial program in one file.
- Reads the same input format as Exercise 1 from standard input: `./barnes_hut < input.dat`.
- Create larger inputs with `exercise_1/random_sphere.f90`.
- Two parameters: the opening angle `theta = 0.5` and the softening length `eps = 0.01`.

Exercise 2: how the starting code works

The particles move with the leapfrog method, as in Exercise 1. After every drift, the accelerations are computed in four steps:

1. Put all the particles into an octree. A node that would hold two particles is divided into eight children.
2. Compute the total mass and the centre of mass of every node, from the leaves up.
3. For each particle, walk down the tree from the root. A node of size s at distance d is treated as a point mass if $s/d < \theta$. Otherwise, visit its children.
4. Delete the tree.

```
type :: node
  real(dp) :: centre(3)
  real(dp) :: size
  integer :: n_particles = 0
  integer :: particle = 0
  real(dp) :: mass = 0.0_dp
  real(dp) :: com(3) = 0.0_dp
  type(node), pointer :: parent => null()
  type(node), pointer :: &
      children(:,:,:) => null()
end type node
```

`particle` is the index of the particle in a leaf, and `com` is the centre of mass. A leaf has no children.

The softened acceleration $m\mathbf{r}/(|\mathbf{r}|^2 + \epsilon^2)^{3/2}$ avoids huge accelerations in close encounters.

Exercise 2: the program

1. Write a makefile for the project.
2. Move the `node` type and the tree procedures into a module `barnes_hut` in the file `barnes_hut.f90`, and the main program into the file `ex2.f90`.
3. Change the code to use the `geometry` and `particle` modules of Exercise 1, and store the particles in an array of `particle3d`.
4. Read the simulation setup from a file whose name is given as a command-line argument, and write the simulation data into `output.dat` in the same format as in Exercise 1.
5. Check your program against your Exercise 1 program. With $\theta = 0$ and $\epsilon = 0$, both compute the same accelerations, so a short run with the same input must give the same positions to many decimal places.

Exercise 2: MPI

6. Write a distributed version of the program with MPI in the file `ex2_mpi.f90`. It uses the same modules. Use this strategy:

Every rank builds the whole tree

- Every rank reads the input and keeps all the particles. The particles are divided into blocks of consecutive particles, one block for each rank.
- In every step, each rank
  1. kicks and drifts the particles of its own block,
  2. gathers the new positions of all the particles from all the ranks with `MPI_Allgatherv`,
  3. builds the whole tree, and computes the accelerations of its own particles only,
  4. kicks its own particles.
- Only rank 0 writes `output.dat`.

Hints: N need not be divisible by the number of ranks, so the blocks can differ in size: `MPI_Allgatherv` takes arrays of counts and displacements. Copy the positions into a `real(dp)` array of shape `(3, n)` before gathering them.

Exercise 2: timing

7. Measure the run times. Create the inputs with `random_sphere.f90` with a seed, run 10 steps, and set `dt_out` equal to `t_end`.
   - Compare your Exercise 1 program (one thread) with the serial Barnes–Hut program for $N = 1000$, 2000, 5000, and 10 000. For which N does Barnes–Hut become faster?
   - Run the MPI program with 1, 2, 4, and 8 ranks for $N = 10\,000$ and 100 000. Compute the speed-ups. Why are they smaller than the number of ranks?
8. Write a `README.md` that explains how the code works, how to compile and run it, and the results of the timing study (in a table or a figure).

Bonus: parallelise the walk of the tree with OpenMP, in the serial or in the MPI program. Each particle's acceleration is written by one thread only, so there is no race condition, but the particles need different amounts of work. Compare the schedules `static` and `dynamic`.

Exercise 2: submission

9. Create a pull request (PR) with your code, and add all the course participants as reviewers.
10. Review the code in the other students' PRs, and write at least one comment on each.
11. (Optional) Revise your code based on the comments you have received.
12. In the PR, tag the teacher and say that you have finished the exercise.

Exercise 2: scoring

Scoring is based on

- Code (80%)
  1. Functionality: the serial and the MPI programs work as described.
  2. Understandability: the code is clear to read and logical.
  3. Comments: all the functionality in the modules and the programs is documented with comments.
  4. Use of guidelines: the code uses the module, type, and file names exactly as written in the exercise.
  5. Timing study: the measurements are complete and explained.
- Collaboration (20%): GitHub PRs, PR comments, and participation.

Bonus points for the OpenMP task, anything else beyond what has been taught in class, frequent commits, and interaction. If in doubt, ask!

Notes: Save all the source files in `students/<initials>/course_exercise_2`. All the real variables must be 64-bit. Compile the MPI program with `mpif90` and run it with `mpirun -np 4 ./ex2_mpi input.dat`.